

DevNet Resource Validation

Use AI to recommend a DevNet resource, verify it against official Cisco documentation, and record the plan in Git.

MODULE	DURATION	MODE	CHECKS
2	90 minutes	Individual or Pair	9

Core rule

An AI recommendation is provisional. Accept it only after an official Cisco source supports the resource type, access model, and intended use case.

Name: Dawn Andrei C. Pamesa

Section: TS31

Course: CS0016, Network and Communications 2

Date submitted: September 13, 2026

AI tool used: Claude (Anthropic)

Files submitted: Pamesa_DawnAndrei_CS0016_M2_DevNetPlan.zip and Pamesa_DawnAndrei_CS0016_M2_DevNetPlan.pdf

1. Overview

You are preparing a development-environment plan for four fictional network-automation tasks. For each task, you will ask an approved AI tool to recommend one DevNet resource category, independently verify the recommendation using official Cisco documentation, complete a structured JSON plan, run an offline validator, and preserve the evidence in a local Git repository.

2. Learning outcomes

- Explain the roles of DevNet Learning Labs, DevNet Sandbox, and Cisco Code Exchange.
- Differentiate Always-On and Reservation Sandboxes based on access, isolation, setup, and administrative privileges.
- Select and justify an appropriate DevNet resource for a stated development or automation need.
- Validate AI-generated claims using official technical documentation.
- Use Git to preserve a reproducible resource-selection and validation record.

3. Required tools and files

Item	Requirement
Browser	Access only official Cisco DevNet pages for evidence
AI tool	Instructor-approved generative AI assistant
Python	Python 3.10 or later; no third-party packages
Version control	Git installed locally; GitHub publishing only when instructed
Starter files	devnet_use_cases.json, student_plan_template.json, validate_plan.py, README_template.md

4. Safety, ethics, and account protection

Read-only exploration by default

Do not create a reservation, connect a VPN, run API requests, or change a sandbox unless the instructor explicitly authorizes it. Never paste usernames, passwords, API keys, tokens, reservation emails, private repository content, or personal data into an AI tool.

- Use only fictional use cases supplied with the activity.
- Official evidence must come from developer.cisco.com.
- Record whether each AI recommendation was verified, partially verified, or rejected.
- A correct selection without evidence and AI disclosure is incomplete.

5. Resource decision guide

Resource	Use when	Important constraint
Learning Lab	Guided, step-by-step learning is the main need	Learning content; not a substitute for choosing an execution environment
Always-On Sandbox	Immediate shared access and non-admin API exploration are sufficient	Shared environment with restricted administrative access
Reservation Sandbox	Private access, administrative control, or configuration testing is required	Reservation, setup time, and often VPN access are required
Cisco Code Exchange	Existing code repositories or automation examples are needed	Code must still be reviewed, tested, and matched to the use case

6. Official evidence sources

- DevNet Sandbox documentation: <https://developer.cisco.com/docs/sandbox/>

- DevNet Learning Labs: <https://developer.cisco.com/learning/>
- Cisco Code Exchange: <https://developer.cisco.com/codeexchange/>

Terminology update

Cisco states that Automation Exchange has been consolidated with Code Exchange. Use Cisco Code Exchange in the revised syllabus and student outputs.

7. Procedure

1. **Prepare the workspace:** Create a folder named it0123-devnet-resource-plan and place all starter files inside it.
2. **Initialize Git:** Run git init, add README_template.md as README.md, and create an initial commit with a meaningful message.
3. **Inspect the scenarios:** Open devnet_use_cases.json. Identify the decisive requirement in each use case before consulting AI.
4. **Prompt AI:** Ask the AI to recommend one of four resource types: learning-lab, always-on-sandbox, reservation-sandbox, or code-exchange. Require a reason and a claim that can be verified.
5. **Verify independently:** Open the relevant official Cisco page. Check the AI claim against the resource description, access model, and intended use.
6. **Complete the plan:** Copy student_plan_template.json as student_plan.json. Fill every field using the exact resource labels required by the validator.
7. **Run validation:** Execute python validate_plan.py. Correct the plan until the validator reports 9/9 checks passed.
8. **Complete the README:** Summarize the decisions, official evidence, AI evaluation, validator result, and AI-use disclosure.
9. **Commit the evidence:** Add the completed JSON, README, and a text copy or screenshot of the validator result. Commit with a message that describes the validation milestone.
10. **Prepare for oral checking:** Be ready to defend one resource decision and explain why an alternative resource was less suitable.

8. Suggested AI prompt

I am selecting a Cisco DevNet resource for a fictional classroom use case.
 Use case: [paste one scenario]
 Choose exactly one: learning-lab, always-on-sandbox,
 reservation-sandbox, or code-exchange.
 Explain which requirement drove your choice. State any access, isolation,
 setup, or privilege claim that I should verify in official Cisco documentation.
 Do not invent a sandbox product name, URL, account, or credential.

9. Required labels

```
learning-lab  
always-on-sandbox  
reservation-sandbox  
code-exchange
```

Verification status: verified | partially-verified | rejected

10. Evidence and deliverables

- student_plan.json with all four validated resource decisions
- README.md with official sources, Git evidence, and AI-use disclosure
- validator output showing 9/9 checks passed
- local Git repository with at least two meaningful commits
- AI prompt record or transcript excerpt for each scenario
- Optional instructor-approved GitHub repository link

11. Reflection questions

1. Which scenario was easiest to classify, and which requirement determined your choice?

UC1 was the easiest one to classify. The scenario already gives away the answer in its last line, which is that the team cannot wait for provisioning. Once that requirement is spotted, the Reservation Sandbox is eliminated immediately, because reserving a sandbox is a waiting step by definition. Learning Labs and Code Exchange were never really in the running either, since the team wants to send actual API requests and not read a tutorial or browse repositories. That leaves the Always-On Sandbox, and the rest of the scenario supports it too, because shared access and read-only requests are exactly the conditions an Always-On environment is built for.

2. Describe one AI claim that needed correction, qualification, or rejection.

The clearest one was UC4. The AI recommended always-on-sandbox and explained it well, saying that the developer could start immediately and run the automation code against real devices. The reasoning sounded fine on its own, however it answered a requirement that the scenario never stated. The scenario asks for community and Cisco-maintained repositories to examine before designing a new solution, so what is needed is existing code, not a device to run it on. I rejected the recommendation and selected code-exchange instead, based on the Code Exchange page describing it as a curated index of those repositories.

A second one worth noting is UC3, where the AI got the resource right but attached a claim that Learning Labs give the learner admin-level devices to keep afterwards. I could not support that, and it also runs against the module's own warning that a Learning Lab is learning content and not a substitute for an execution environment, so I recorded it as partially-verified rather than just accepting the whole answer because the label happened to be correct.

3. Why is an Always-On Sandbox not automatically the best option for every API task?

Because the things that make it convenient are the same things that limit it. An Always-On Sandbox is shared and immediately reachable, which is great when the task is read-only exploration, yet the sharing is precisely

the problem once the task involves configuration. It is a shared environment with restricted administrative access, so a configuration change would either be blocked outright or, worse, would land on an environment other people are using at the same time. There is also no isolation to speak of, which means a test cannot be assumed to be clean or repeatable, since somebody else may be working in the same place. For anything that needs administrative control, a private environment, or a predictable starting state, the Reservation Sandbox is the correct choice even though it costs scheduling and setup time.

I realized while doing this that “fastest to access” and “most appropriate” are two different questions, and the second one is the one being graded.

4. How did Git improve the traceability of your resource-selection process?

Git made the process reviewable instead of just the final answer being reviewable. The first commit captured the workspace before any decision existed, and the validation commit captured the finished plan together with the validator output, so the two states can be compared and it is visible what was actually decided at which point rather than only what the file looks like now. Because the prompt record and the verification statuses are committed alongside the plan, anyone checking the work can see which AI claims were accepted, which were qualified, and which one was thrown out, and they can see that the rejection happened before the plan was validated and not afterwards as a cleanup.

To conclude, the commits turn the plan from a single submitted file into a record that shows the order the work was done in, which is the part that matters when the requirement is to prove that an AI recommendation was verified and not just copied.

12. Assessment rubric

Criterion	Points	Evidence
Resource classification	25	Four selections match the scenario requirements
Official evidence validation	25	Accurate Cisco sources and defensible verification
AI evaluation	20	Recommendations summarized and critically accepted, modified, or rejected
Plan and Git workflow	20	Complete JSON/README, passing validator, meaningful commits
Ethics and professionalism	10	No credentials exposed; clear and complete submission
TOTAL	100	

Submission filename: Surname_FirstName_IT0123_M2_DevNetPlan.zip